

I2C Driver Documentation

1. Driver Overview

The I2C driver provides a hardware abstraction layer for the Inter-Integrated Circuit (I2C) peripheral of the STM32F401 microcontroller. The driver allows an application to configure and use the I2C peripheral without directly accessing hardware registers.

The driver supports:

- I2C peripheral initialization
- Clock configuration
- Master transmission and reception
- Slave transmission and reception
- Blocking (polling) communication
- Interrupt-driven communication
- ACK management
- Event interrupt handling
- Error interrupt handling
- NVIC interrupt configuration

The driver hides low-level register operations and provides a clean API for communication with I2C devices such as EEPROMs, RTCs, sensors, and displays.

2. Driver Architecture

Application Layer

↓

I2C Driver APIs

↓

I2C Peripheral Registers

(CR1, CR2, SR1, SR2, DR, CCR, TRISE, OAR1)

↓

The application interacts only through driver APIs and configuration structures. All register manipulation is performed internally by the driver.

3. Initialization Process

The I2C_Init() API configures the peripheral using parameters supplied through the I2C_Handle_t structure.

Configuration steps:

1. Enable peripheral clock.
2. Configure ACK control.
3. Configure CR2 frequency field.
4. Program own slave address.
5. Calculate CCR value.
6. Configure Standard Mode or Fast Mode operation.
7. Configure TRISE value.

The driver automatically calculates timing values based on the peripheral clock frequency and selected I2C speed.

4. I2C Communication Flow

Master Transmission

The driver performs the following sequence:

1. Generate START condition.
2. Wait for SB flag.
3. Send slave address with Write bit.
4. Wait for ADDR flag.
5. Clear ADDR flag.
6. Send data bytes.
7. Wait for TXE and BTF flags.
8. Generate STOP condition.

Master Reception

The driver performs:

1. Generate START condition.
2. Wait for SB flag.
3. Send slave address with Read bit.
4. Wait for ADDR flag.
5. Configure ACK/NACK handling.
6. Receive data bytes.
7. Generate STOP condition.
8. Restore ACK configuration.

Special handling is implemented for:

- Single-byte reception
- Multi-byte reception
- Last-byte NACK generation

5. Blocking Communication APIs

These APIs use polling and wait until communication completes.

I2C_MasterSendData()

Purpose

Transmit a complete data buffer to a slave device.

Internal Operation

Uses:

- SB flag
- ADDR flag
- TXE flag

- BTF flag
- Data Register (DR)

Abstraction

Provides a single API for complete data transmission without requiring the application to monitor hardware flags.

I2C_MasterReceiveData()

Purpose

Receive a complete data buffer from a slave device.

Internal Operation

Uses:

- SB flag
- ADDR flag
- RXNE flag
- ACK/NACK management

Abstraction

Provides complete receive functionality while hiding reception timing and ACK control details.

I2C_SlaveSendData()

Purpose

Transmit one byte from slave device.

Abstraction

Application only provides data while driver handles register access.

I2C_SlaveReceiveData()

Purpose

Read one byte received by slave device.

Abstraction

Returns received data without exposing register operations.

6. Interrupt Driven Communication

The driver supports non-blocking communication using interrupts.

I2C_MasterSendDataIT()

Purpose

Start interrupt-driven transmission.

Internal Actions

- Stores transmit buffer information.
- Marks peripheral busy.
- Generates START condition.
- Enables event, buffer and error interrupts.

Abstraction

Application starts transmission using a single API while the driver handles data transfer in interrupt context.

I2C_MasterReceiveDataIT()

Purpose

Start interrupt-driven reception.

Internal Actions

- Stores receive buffer information.
- Marks peripheral busy.
- Generates START condition.
- Enables interrupt sources.

Abstraction

Application does not need to manage receive timing or hardware events.

7. Interrupt Event Handling

I2C_EV_IRQHandling()

This function acts as the central event interrupt handler.

The handler processes:

SB Event

Triggered after START generation.

Action:

- Sends slave address.
-

ADDR Event

Triggered after address transmission.

Action:

- Clears ADDR flag.
-

BTF Event

Triggered when byte transfer completes.

Action:

- Finishes transmission.
 - Generates STOP condition.
 - Notifies application.
-

STOPF Event

Triggered in slave mode when STOP condition is detected.

Action:

- Clears STOP flag.
 - Notifies application.
-

TXE Event

Triggered when transmit register becomes empty.

Action:

- Loads next byte into DR.
-

RXNE Event

Triggered when data becomes available.

Action:

- Reads data from DR.
 - Handles final-byte reception.
 - Generates STOP condition.
 - Notifies application.
-

8. Error Interrupt Handling

I2C_ER_IRQHandling()

Handles I2C communication errors.

Bus Error (BERR)

Occurs due to invalid START or STOP conditions.

Driver Actions:

- Clears error flag.
 - Generates STOP.
 - Closes active transaction.
 - Notifies application.
-

Arbitration Lost (ARLO)

Occurs in multi-master systems.

Driver Actions:

- Clears flag.
 - Closes transaction.
 - Notifies application.
-

Acknowledge Failure (AF)

Occurs when receiver does not acknowledge transmitted data.

Driver Actions:

- Clears flag.
- Generates STOP.
- Closes transaction.
- Notifies application.

Overflow / Underflow (OVR)

Occurs when data is not read or written in time.

Driver Actions:

- Clears flag.
- Closes transaction.
- Notifies application.

Timeout Error

Occurs mainly in SMBus operation.

Driver Actions:

- Clears flag.
- Closes transaction.
- Notifies application.

9. Peripheral Control APIs

I2C_PeripheralControl()

Purpose

Enable or disable the I2C peripheral.

Internal Register

CR1.PE

Abstraction

Provides peripheral control without direct register access.

I2C_PeriClockControl()

Purpose

Enable or disable I2C clock.

Internal Register

RCC APB1ENR

Abstraction

Hides RCC register manipulation.

I2C_DeInit()

Purpose

Reset I2C peripheral.

Internal Register

RCC APB1RSTR

Abstraction

Provides simple peripheral reset functionality.

I2C_ManageAcking()

Purpose

Enable or disable ACK generation.

Internal Register

CR1.ACK

Abstraction

Provides ACK control through a dedicated API.

10. Interrupt Configuration APIs

I2C_IRQInterruptConfig()

Purpose

Enable or disable NVIC interrupts.

Internal Registers

- ISER
- ICER

Abstraction

Hides NVIC register programming.

I2C_IRQPriorityConfig()

Purpose

Configure interrupt priority.

Internal Registers

- IPR registers

Abstraction

Provides simple interrupt priority configuration.

11. Internal Driver Functions

The following functions are internal helper functions and are not intended for direct application use:

- I2C_GenerateStartCondition()
- I2C_GenerateStopCondition()
- I2C_ExecuteAddressPhaseWrite()
- I2C_ExecuteAddressPhaseRead()
- I2C_ClearADDRFlag()
- I2C_MasterHandleTXEInterrupt()
- I2C_MasterHandleRXNEInterrupt()
- I2C_CloseSendData()
- I2C_CloseReceiveData()

These functions implement low-level protocol operations and support the public APIs.

12. Abstraction Provided by the Driver

The driver hides the following hardware details:

- RCC clock control registers
- I2C timing calculations
- START and STOP generation
- Address phase handling
- ACK/NACK management
- Status flag monitoring
- Interrupt enable registers
- NVIC configuration
- Error recovery mechanisms

The application only interacts through high-level APIs and callback events.

13. Conclusion

The I2C driver provides a complete abstraction layer for the STM32F401 I2C peripheral. It supports both polling-based and interrupt-driven communication while handling protocol details such as address transmission, ACK management, interrupt servicing, and error recovery internally. This approach improves code modularity, maintainability, and reusability while keeping the application independent of hardware register operations.

